

# Python Deep-Dive Study Notes

| Concepts · Practical Code · Hands-on Exercises

- 1 Introduction to Python
- 2 Variables and Data Types
- 3 Operators and Expressions
- 4 Conditional Statements
- 5 Loops and Iterations
- 6 Functions and Modules
- 7 File Handling
- 8 Exception Handling
- 9 Object-Oriented Programming
- 10 NumPy Fundamentals
- 11 Pandas Fundamentals

# 01

## Introduction to Python

What Python is, why it's popular, how it runs, and writing your first programs.

# 1. Introduction to Python

## 1.1 What is Python?

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. It emphasizes code readability through significant indentation and a clean, English-like syntax.

KEY CHARACTERISTICS

- **Interpreted:** code runs line-by-line without a separate compilation step
- **Dynamically typed:** you don't declare variable types; Python infers them at runtime
- **High-level:** manages memory and low-level details automatically
- **Multi-paradigm:** supports procedural, object-oriented, and functional styles
- **Batteries included:** a huge standard library plus a massive third-party ecosystem (PyPI)

## 1.2 Why Python is Popular

| Domain                 | Why Python fits                                           |
|------------------------|-----------------------------------------------------------|
| Data Science / ML      | NumPy, Pandas, scikit-learn, TensorFlow, PyTorch          |
| Web Development        | Django, Flask, FastAPI                                    |
| Automation / Scripting | Simple syntax, cross-platform, great for quick scripts    |
| Education              | Readable syntax lowers the barrier to entry for beginners |

## 1.3 Installing Python & Running Code

You can write Python code in three main ways:

1. **Interactive Shell (REPL):** type `python3` in a terminal and run statements one at a time
2. **Script files:** save code in a `.py` file and run it with `python3 filename.py`
3. **Notebooks:** Jupyter notebooks mix code, output, and text — popular for data analysis

CODE — your first program

```
# hello.py
print("Hello, World!")
print("Welcome to Python")

# Run it from the terminal with:
# python3 hello.py
```

OUTPUT

```
Hello, World!
Welcome to Python
```

## 1.4 Comments and Indentation

Python uses `#` for single-line comments and triple quotes for multi-line comments/docstrings. Unlike many languages, Python uses **indentation** (not curly braces) to define code blocks.

CODE

```
# This is a single-line comment
"""
This is a multi-line comment
or a docstring used to describe a function/module
"""
```

```
if True:
    print("This line is indented — it belongs to the if block")
print("This line is not indented — runs regardless")
```

### COMMON PITFALL

Mixing tabs and spaces for indentation causes an `IndentationError`. Stick to 4 spaces per level (the PEP 8 standard) and configure your editor accordingly.

## 1.5 Python 2 vs Python 3

Python 2 reached end-of-life in January 2020. All modern development uses Python 3. Key differences include `print` being a function (not a statement), integer division behavior, and Unicode-by-default strings in Python 3.

### Practice Exercises — Introduction to Python

- EASY** Write a script that prints your name, age, and favorite programming language on three separate lines using three separate `print()` statements.
- EASY** Write a one-line comment and a multi-line docstring comment above a `print()` statement that prints "Learning Python is fun!"
- MEDIUM** Predict the output of the following code without running it, then verify by running it:

```
if 5 > 3:
    print("A")
    print("B")
print("C")
```

(Hint: think about what an indentation error looks like and where it occurs.)

# 02

## **Variables and Data Types**

Storing data, naming rules, and Python's built-in types.

## 2. Variables and Data Types

### 2.1 Variables

A variable is a name bound to a value in memory. Python variables don't need explicit type declarations — the type is determined by the value assigned.

#### CODE

```
name = "Alice"      # string
age = 25            # integer
height = 5.6        # float
is_student = True   # boolean

print(name, age, height, is_student)
print(type(name), type(age), type(height), type(is_student))
```

#### OUTPUT

```
Alice 25 5.6 True
<class 'str'> <class 'int'> <class 'float'> <class 'bool'>
```

#### VARIABLE NAMING RULES

- Must start with a letter or underscore (not a digit)
- Can contain letters, digits, and underscores only — no spaces or special characters
- Case-sensitive: `age` and `Age` are different variables
- Cannot use reserved keywords like `if`, `for`, `class`
- Convention: use `snake_case` for variable and function names

### 2.2 Built-in Data Types

| Type                  | Example      | Description                            |
|-----------------------|--------------|----------------------------------------|
| <code>int</code>      | 10, -5, 1000 | Whole numbers, unlimited precision     |
| <code>float</code>    | 3.14, -0.5   | Decimal/floating-point numbers         |
| <code>str</code>      | "hello"      | Immutable sequence of characters       |
| <code>bool</code>     | True, False  | Boolean truth values (subclass of int) |
| <code>list</code>     | [1, 2, 3]    | Ordered, mutable collection            |
| <code>tuple</code>    | (1, 2, 3)    | Ordered, immutable collection          |
| <code>dict</code>     | {"a": 1}     | Key-value pairs, mutable               |
| <code>set</code>      | {1, 2, 3}    | Unordered collection of unique items   |
| <code>NoneType</code> | None         | Represents "no value"                  |

### 2.3 Type Conversion (Casting)

#### CODE

```
x = "100"
y = int(x)      # string -> int
z = float(y)    # int -> float
s = str(z)      # float -> string

print(y, type(y))
print(z, type(z))
```

```
print(s, type(s))

# Be careful: this raises a ValueError
# int("hello") -> ValueError: invalid literal for int()
```

## OUTPUT

```
100 <class 'int'>
100.0 <class 'float'>
100.0 <class 'str'>
```

## 2.4 Strings in Depth

### CODE

```
text = "Python Programming"

print(text.upper())      # PYTHON PROGRAMMING
print(text.lower())      # python programming
print(text.replace("Python", "Java")) # Java Programming
print(text.split())      # ['Python', 'Programming']
print(text[0:6])         # Python (slicing)
print(len(text))         # 19

# f-strings (formatted string literals) — the modern way to format
name, score = "Riya", 92.5
print(f"{name} scored {score:.1f}%")
```

### OUTPUT

```
PYTHON PROGRAMMING
python programming
Java Programming
['Python', 'Programming']
Python
19
Riya scored 92.5%
```

### TIP

Strings are **immutable** — methods like `.upper()` return a *new* string rather than modifying the original in place.

## 2.5 Lists, Tuples, Dictionaries, Sets — Quick Look

### CODE

```
fruits = ["apple", "banana", "cherry"] # list: ordered, mutable
coords = (10.5, 20.3)                  # tuple: ordered, immutable
student = {"name": "Sam", "age": 21}   # dict: key-value pairs
unique_nums = {1, 2, 2, 3}             # set: duplicates removed

fruits.append("date")
print(fruits)      # ['apple', 'banana', 'cherry', 'date']
print(student["name"]) # Sam
print(unique_nums)  # {1, 2, 3}
```

### Practice Exercises — Variables and Data Types

1. **EASY** Create variables for your favorite movie's title (str), release year (int), rating out of 10 (float), and whether you'd recommend it (bool). Print each with its type.
2. **EASY** Given `a = "42"`, convert it to an integer, add 8 to it, then convert the result back to a string and print the final value with its type.
3. **MEDIUM** Write a program that takes the string `" Python is Awesome "` and produces, on separate lines: the string with whitespace stripped, the string in all lowercase, and the number of vowels in it.

4. **HARD** Create a dictionary representing a product with keys `name`, `price`, and `tags` (a list). Write code to add a new tag, update the price by applying a 10% discount, and print a formatted sentence describing the final product using an f-string.



# 03

## Operators and Expressions

Arithmetic, comparison, logical, assignment, and membership operators.

## 3. Operators and Expressions

### 3.1 Arithmetic Operators

| Operator        | Meaning                      | Example             | Result |
|-----------------|------------------------------|---------------------|--------|
| <code>+</code>  | Addition                     | <code>7 + 3</code>  | 10     |
| <code>-</code>  | Subtraction                  | <code>7 - 3</code>  | 4      |
| <code>*</code>  | Multiplication               | <code>7 * 3</code>  | 21     |
| <code>/</code>  | True division (always float) | <code>7 / 2</code>  | 3.5    |
| <code>//</code> | Floor division               | <code>7 // 2</code> | 3      |
| <code>%</code>  | Modulus (remainder)          | <code>7 % 2</code>  | 1      |
| <code>**</code> | Exponentiation               | <code>2 ** 5</code> | 32     |

#### CODE

```
a, b = 17, 5
print(a + b, a - b, a * b)    # 22 12 85
print(a / b)                  # 3.4
print(a // b)                  # 3
print(a % b)                   # 2
print(a ** 2)                  # 289
```

### 3.2 Comparison Operators

These return a boolean (`True` / `False`): `==`, `!=`, `>`, `<`, `>=`, `<=`

#### CODE

```
x, y = 10, 20
print(x == y)    # False
print(x != y)    # True
print(x < y)     # True
print(x >= 10)   # True
```

### 3.3 Logical Operators

#### CODE

```
age = 25
has_id = True

print(age >= 18 and has_id)    # True - both must be true
print(age < 18 or has_id)      # True - at least one is true
print(not has_id)               # False - inverts the boolean
```

### 3.4 Assignment Operators

#### CODE

```
count = 10
count += 5    # count = count + 5 -> 15
count -= 3    # -> 12
count *= 2    # -> 24
count /= 5    # -> 4
print(count)  # 4
```

## 3.5 Identity and Membership Operators

### CODE

```
nums = [1, 2, 3]
print(2 in nums)      # True  - membership
print(5 not in nums)   # True

a = [1, 2]
b = [1, 2]
print(a == b)         # True  - same VALUES
print(a is b)         # False - different objects in memory
```

### == vs is

`==` compares **values**. `is` compares **identity** (whether two variables point to the exact same object in memory). Use `is` primarily for comparing against `None`: `if x is None:`

## 3.6 Operator Precedence

Python follows mathematical precedence rules (PEMDAS-like): parentheses first, then exponentiation, then multiplication/division, then addition/subtraction, and comparisons/logical operators last.

### CODE

```
result = 2 + 3 * 4 ** 2  # ** first, then *, then +
print(result)           # 2 + 3*16 = 2 + 48 = 50

result2 = (2 + 3) * 4    # parentheses override default order
print(result2)          # 20
```

### Practice Exercises — Operators and Expressions

- EASY** Given two numbers entered by a user, write expressions to print their sum, difference, product, quotient, floor division, remainder, and the first number raised to the power of the second.
- EASY** Write a boolean expression that checks if a number `n` is between 1 and 100 (inclusive) using logical operators.
- MEDIUM** Without running the code, evaluate: `10 + 2 * 3 ** 2 - (4 / 2)`. Then verify with Python.
- HARD** Write a single boolean expression (no if statements) that determines whether a given year is a leap year. (A year is a leap year if divisible by 4, except centuries, which must be divisible by 400.)

# 04

## Conditional Statements

Decision making with `if`, `elif`, `else`, and the ternary expression.

## 4. Conditional Statements

### 4.1 The if Statement

#### CODE

```
age = 20

if age >= 18:
    print("You are an adult.")
```

#### OUTPUT

You are an adult.

### 4.2 if-else

#### CODE

```
temperature = 15

if temperature > 25:
    print("It's warm.")
else:
    print("It's cool.")
```

#### OUTPUT

It's cool.

### 4.3 if-elif-else Chains

#### CODE

```
marks = 72

if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 60:
    grade = "C"
else:
    grade = "F"

print(f"Grade: {grade}")
```

#### OUTPUT

Grade: C

#### HOW IT EVALUATES

Python checks conditions top to bottom and executes the **first** block whose condition is True, then skips the rest of the chain entirely — even if a later condition would also be True.

### 4.4 Nested Conditionals

#### CODE

```
username = "admin"
password = "1234"

if username == "admin":
    if password == "1234":
        print("Login successful")
    else:
        print("Wrong password")
else:
    print("Unknown user")
```

## 4.5 Ternary (Conditional) Expressions

A compact one-line form of if-else, useful for simple assignments.

### CODE

```
age = 16
status = "Adult" if age >= 18 else "Minor"
print(status)  # Minor
```

## 4.6 Truthy and Falsy Values

Every object in Python has an implicit boolean value when used in a condition.

| Falsy             | Truthy                |
|-------------------|-----------------------|
| 0, 0.0            | any non-zero number   |
| "" (empty string) | any non-empty string  |
| [], (), {}, set() | non-empty collections |
| None, False       | True                  |

### CODE

```
cart = []
if cart:
    print("You have items in your cart")
else:
    print("Your cart is empty")  # this runs — empty list is falsy
```

### COMMON PITFALL

Don't confuse `=` (assignment) with `==` (comparison) inside conditions — `if x = 5:` is a `SyntaxError` in Python (unlike some other languages).

### Practice Exercises — Conditional Statements

- EASY** Write a program that checks if a number is positive, negative, or zero.
- EASY** Write a program that determines if a given number is even or odd using the modulus operator.
- MEDIUM** Write a simple ticket-pricing program: free for age < 5, \$10 for age 5–17, \$15 for age 18–64, and \$8 for age 65+. Use if-elif-else.
- HARD** Write a program that takes three side lengths and determines whether they can form a triangle (sum of any two sides must exceed the third), and if so, whether it's equilateral, isosceles, or scalene.

# 05

## **Loops and Iterations**

for, while, break, continue, and looping over data structures.

## 5. Loops and Iterations

### 5.1 The for Loop

Used to iterate over a sequence (list, string, tuple, dict, range, etc.).

#### CODE

```
for i in range(5):  
    print(i)
```

#### OUTPUT

```
0  
1  
2  
3  
4
```

#### CODE — range() with start, stop, step

```
for i in range(2, 11, 2):    # start=2, stop=11 (exclusive), step=2  
    print(i, end=" ")
```

#### OUTPUT

```
2 4 6 8 10
```

#### CODE — iterating collections

```
fruits = ["apple", "banana", "cherry"]  
for fruit in fruits:  
    print(fruit.upper())  
  
# with index using enumerate()  
for idx, fruit in enumerate(fruits, start=1):  
    print(f"{idx}. {fruit}")
```

#### OUTPUT

```
APPLE  
BANANA  
CHERRY  
1. apple  
2. banana  
3. cherry
```

### 5.2 The while Loop

Repeats as long as a condition stays True. Good when you don't know the number of iterations in advance.

#### CODE

```
count = 0  
while count < 5:  
    print(f"Count is {count}")  
    count += 1
```

#### OUTPUT

```
Count is 0
```



```
Count is 1
Count is 2
Count is 3
Count is 4
```

### COMMON PITFALL — Infinite Loops

Forgetting to update the loop variable inside a `while` loop (e.g. forgetting `count += 1`) causes the loop to run forever. Always make sure the condition will eventually become False.

## 5.3 break, continue, and else on Loops

### CODE

```
# break — exit the loop immediately
for num in range(10):
    if num == 5:
        break
    print(num, end=" ")
print()

# continue — skip to the next iteration
for num in range(10):
    if num % 2 == 0:
        continue
    print(num, end=" ")
```

### OUTPUT

```
0 1 2 3 4
1 3 5 7 9
```

## 5.4 Nested Loops

### CODE — multiplication table

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i*j:3}", end=" ")
    print() # newline after each row
```

### OUTPUT

```
1  2  3
2  4  6
3  6  9
```

## 5.5 List Comprehensions (Compact Loops)

A concise, "Pythonic" way to build lists from iterables.

### CODE

```
squares = [x**2 for x in range(1, 6)]
print(squares)           # [1, 4, 9, 16, 25]

evens = [x for x in range(20) if x % 2 == 0]
print(evens)             # [0, 2, 4, ..., 18]
```

### Practice Exercises — Loops and Iterations

- EASY** Print all numbers from 1 to 50 that are divisible by 3 using a `for` loop.
- EASY** Use a `while` loop to compute the factorial of a number entered by the user.

3. **MEDIUM** Write a program that prints the Fibonacci sequence up to n terms using a loop.
4. **MEDIUM** Use a list comprehension to create a list of all vowels found in a given sentence.
5. **HARD** Write a number-guessing game: the program picks a random number between 1 and 100 (use the `random` module), and the user keeps guessing until correct. After each guess, print "Too high" or "Too low". Track and print the number of attempts at the end.

# 06

## Functions and Modules

Reusable code blocks, parameters, scope, lambdas, and importing modules.

## 6. Functions and Modules

### 6.1 Defining and Calling Functions

#### CODE

```
def greet(name):  
    """Returns a greeting message for the given name."""  
    return f"Hello, {name}!"  
  
message = greet("Aditi")  
print(message)
```

#### OUTPUT

```
Hello, Aditi!
```

### 6.2 Parameters: Default, Positional, Keyword, \*args, \*\*kwargs

#### CODE

```
def describe_pet(name, animal="dog"):    # default value  
    print(f"{name} is a {animal}")  
  
describe_pet("Rex")                    # uses default "dog"  
describe_pet("Whiskers", animal="cat") # keyword argument  
  
def add_all(*args):                    # *args -> variable positional args (tuple)  
    return sum(args)  
  
print(add_all(1, 2, 3, 4))             # 10  
  
def print_info(**kwargs):              # **kwargs -> variable keyword args (dict)  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")  
  
print_info(name="Sam", age=22, city="Pune")
```

#### OUTPUT

```
Rex is a dog  
Whiskers is a cat  
10  
name: Sam  
age: 22  
city: Pune
```

### 6.3 Return Values

A function without an explicit `return` implicitly returns `None`. Functions can return multiple values as a tuple.

#### CODE

```
def min_max(numbers):  
    return min(numbers), max(numbers)  
  
low, high = min_max([4, 9, 1, 7])  
print(f"Min: {low}, Max: {high}")    # Min: 1, Max: 9
```

### 6.4 Variable Scope

**CODE**

```
x = 10    # global scope

def modify():
    x = 5    # local scope — this is a NEW variable, shadows global x
    print("Inside:", x)

modify()
print("Outside:", x)    # global x is unchanged

def modify_global():
    global x
    x = 99    # this DOES change the global variable

modify_global()
print("After global change:", x)
```

**OUTPUT**

```
Inside: 5
Outside: 10
After global change: 99
```

## 6.5 Lambda (Anonymous) Functions

**CODE**

```
square = lambda x: x ** 2
print(square(6))    # 36

# commonly used with map(), filter(), sorted()
nums = [5, 3, 8, 1]
print(sorted(nums, key=lambda n: -n))    # [8, 5, 3, 1] descending
print(list(map(lambda n: n*2, nums)))    # [10, 6, 16, 2]
```

## 6.6 Modules and Imports

A module is simply a `.py` file containing reusable code. Python's standard library ships with many useful modules.

**CODE**

```
import math
import random as rnd
from datetime import datetime

print(math.sqrt(144))    # 12.0
print(rnd.randint(1, 10))    # random int between 1 and 10
print(datetime.now().year)    # current year

# Creating your own module:
# save the following in a file named "calculator.py":
#     def add(a, b):
#         return a + b
# then in another file:
#     import calculator
#     print(calculator.add(3, 4))
```

**TIP**

Use `if __name__ == "__main__":` at the bottom of a script so its top-level code only runs when executed directly — not when imported as a module elsewhere.

### Practice Exercises — Functions and Modules

1. **EASY** Write a function `is_prime(n)` that returns True if `n` is a prime number, otherwise False.

2. **EASY** Write a function `average(*numbers)` that accepts any number of arguments and returns their average using `*args`.
3. **MEDIUM** Write a function `temperature_converter(value, to_unit="celsius")` that converts between Celsius and Fahrenheit based on a keyword argument.
4. **MEDIUM** Create your own module file called `string_utils.py` with functions `is_palindrome(s)` and `count_words(s)`, then import and use them in a separate script.
5. **HARD** Write a function `apply_discount` that takes a list of prices and a lambda function as a parameter (e.g., a 10% discount rule), and returns a new list of discounted prices using `map()`.

# 07

## File Handling

Reading, writing, and managing files safely with context managers.

# 7. File Handling

## 7.1 Opening Files: Modes

| Mode        | Meaning                                             |
|-------------|-----------------------------------------------------|
| "r"         | Read (default) — file must exist                    |
| "w"         | Write — creates file or overwrites existing content |
| "a"         | Append — adds to the end of the file                |
| "x"         | Create — fails if file already exists               |
| "r+"        | Read and write                                      |
| "rb" / "wb" | Binary read / write (for non-text files)            |

## 7.2 Writing to a File

### CODE

```
with open("notes.txt", "w") as f:
    f.write("Python is fun.\n")
    f.write("File handling is easy.\n")
# file is automatically closed when the 'with' block ends
```

### WHY USE with

The `with` statement (a context manager) guarantees the file is properly closed even if an error occurs inside the block — far safer than manually calling `f.close()`.

## 7.3 Reading from a File

### CODE

```
# Read entire file as one string
with open("notes.txt", "r") as f:
    content = f.read()
    print(content)

# Read line by line (memory-efficient for large files)
with open("notes.txt", "r") as f:
    for line in f:
        print(line.strip()) # strip() removes the trailing newline

# Read all lines into a list
with open("notes.txt", "r") as f:
    lines = f.readlines()
    print(lines)
```

### OUTPUT

```
Python is fun.
File handling is easy.

Python is fun.
File handling is easy.
['Python is fun.\n', 'File handling is easy.\n']
```

## 7.4 Appending to a File



**CODE**

```
with open("notes.txt", "a") as f:
    f.write("This line was appended.\n")
```

## 7.5 Working with CSV Files

**CODE**

```
import csv

# Writing a CSV
with open("students.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Name", "Score"])
    writer.writerow(["Asha", 88])
    writer.writerow(["Ravi", 91])

# Reading a CSV
with open("students.csv", "r") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

**OUTPUT**

```
['Name', 'Score']
['Asha', '88']
['Ravi', '91']
```

## 7.6 Checking If a File Exists

**CODE**

```
import os

if os.path.exists("notes.txt"):
    print("File found!")
else:
    print("File does not exist.")
```

**COMMON PITFALL**

Opening a file that doesn't exist in `"r"` mode raises a `FileNotFoundError`. Always validate the path exists, or wrap the operation in a try-except block (covered in the next section).

### Practice Exercises — File Handling

1. **EASY** Write a program that creates a file called `diary.txt` and writes three lines of text to it, then reads and prints the whole file.
2. **EASY** Write a program that counts the number of lines, words, and characters in a text file.
3. **MEDIUM** Write a program that reads a CSV file of products (name, price, quantity) and prints the total inventory value (sum of price × quantity for all rows).
4. **HARD** Write a simple "append-only" logging function `log_event(message)` that appends a timestamped message to `app_log.txt` each time it's called, creating the file if it doesn't already exist.

# 08

## Exception Handling

try, except, else, finally, raising custom exceptions, and best practices.

## 8. Exception Handling

### 8.1 Why Handle Exceptions?

An exception is an error detected during execution. Without handling, an exception crashes the program. Exception handling lets you respond gracefully instead.

### 8.2 try-except Basics

#### CODE

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print(f"Result: {result}")
except ZeroDivisionError:
    print("You can't divide by zero!")
except ValueError:
    print("That's not a valid number!")
```

#### OUTPUT (if user enters 0)

```
You can't divide by zero!
```

### 8.3 Catching Multiple Exceptions & the Generic Exception

#### CODE

```
try:
    data = [1, 2, 3]
    print(data[10])
except (IndexError, KeyError) as e:
    print(f"Lookup error: {e}")
except Exception as e:
    print(f"Unexpected error: {e}")
```

#### OUTPUT

```
Lookup error: list index out of range
```

#### TIP

Catch the **most specific** exception types first, and place generic `except Exception` last — it acts as a catch-all safety net. Avoid bare `except:` which silently catches everything, including `KeyboardInterrupt`.

### 8.4 else and finally

#### CODE

```
try:
    f = open("data.txt", "r")
    content = f.read()
except FileNotFoundError:
    print("File not found.")
else:
    print("File read successfully.") # runs only if NO exception occurred
finally:
    print("Cleanup complete.") # ALWAYS runs, error or not
```

## 8.5 Raising Your Own Exceptions

### CODE

```
def withdraw(balance, amount):
    if amount > balance:
        raise ValueError("Insufficient funds")
    return balance - amount

try:
    withdraw(100, 500)
except ValueError as e:
    print(f"Transaction failed: {e}")
```

### OUTPUT

Transaction failed: Insufficient funds

## 8.6 Custom Exception Classes

### CODE

```
class InsufficientFundsError(Exception):
    """Raised when a withdrawal exceeds the account balance."""
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(f"Tried to withdraw {amount}, only {balance} available")
    return balance - amount

try:
    withdraw(50, 200)
except InsufficientFundsError as e:
    print("Custom exception caught:", e)
```

## 8.7 Common Built-in Exceptions

| Exception                      | Triggered When                                            |
|--------------------------------|-----------------------------------------------------------|
| <code>ValueError</code>        | Right type, invalid value (e.g. <code>int("abc")</code> ) |
| <code>TypeError</code>         | Operation on incompatible types                           |
| <code>IndexError</code>        | Sequence index out of range                               |
| <code>KeyError</code>          | Dictionary key not found                                  |
| <code>ZeroDivisionError</code> | Division or modulus by zero                               |
| <code>FileNotFoundError</code> | Opening a non-existent file in read mode                  |
| <code>AttributeError</code>    | Accessing a non-existent attribute/method                 |

### Practice Exercises — Exception Handling

- EASY** Write a program that asks the user for two numbers and divides them, handling both `ValueError` (non-numeric input) and `ZeroDivisionError`.
- EASY** Write a function that safely accesses a dictionary key, catching `KeyError` and returning a default value of `"Not Found"` instead.
- MEDIUM** Create a custom exception `NegativeAgeError` and write a function `set_age(age)` that raises it when given a negative number.
- HARD** Write a robust input function `get_positive_integer(prompt)` that keeps asking the user for input in a loop until they provide a valid positive integer, using try-except to catch invalid entries (non-numbers).

and negative numbers) without crashing.

# 09

## Object-Oriented Programming

Classes, objects, inheritance, encapsulation, and polymorphism.

# 9. Object-Oriented Programming Basics

## 9.1 Classes and Objects

A **class** is a blueprint for creating objects. An **object** (or instance) is a specific realization of that blueprint with its own data.

### CODE

```
class Dog:
    def __init__(self, name, breed):
        self.name = name      # instance attribute
        self.breed = breed

    def bark(self):           # instance method
        return f"{self.name} says Woof!"

d1 = Dog("Rex", "Labrador")
d2 = Dog("Bella", "Poodle")

print(d1.bark())            # Rex says Woof!
print(d2.breed)             # Poodle
```

### `__init__` and `self`

`__init__` is the constructor — it runs automatically when an object is created. `self` refers to the specific instance calling the method and must be the first parameter of every instance method.

## 9.2 Class Attributes vs Instance Attributes

### CODE

```
class Employee:
    company = "TechCorp"      # class attribute — shared by ALL instances

    def __init__(self, name, salary):
        self.name = name      # instance attribute — unique per object
        self.salary = salary

e1 = Employee("Nina", 55000)
e2 = Employee("Omar", 62000)
print(e1.company, e2.company) # TechCorp TechCorp
```

## 9.3 Encapsulation

Encapsulation bundles data and methods together, and restricts direct access using naming conventions: single underscore `protected` (convention, not enforced) and double underscore `private` (name-mangled).

### CODE

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    # private attribute

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):            # controlled access via a "getter"
        return self.__balance

acc = BankAccount(1000)
acc.deposit(500)
print(acc.get_balance())            # 1500
# print(acc.__balance)              -> AttributeError, direct access blocked
```

## 9.4 Inheritance

### CODE

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return f"{self.name} makes a sound"

class Cat(Animal):
    # Cat inherits from Animal
    def speak(self):
        # method overriding
        return f"{self.name} says Meow"

class Bird(Animal):
    def speak(self):
        return f"{self.name} says Tweet"

animals = [Cat("Tom"), Bird("Tweety"), Animal("Generic")]
for a in animals:
    print(a.speak())
```

### OUTPUT

```
Tom says Meow
Tweety says Tweet
Generic makes a sound
```

## POLYMORPHISM

The loop above is polymorphism in action: calling `.speak()` on each object runs *that object's own version* of the method, even though they're all treated as "Animal" in the loop.

## 9.5 super() — Calling the Parent's Method

### CODE

```
class Vehicle:
    def __init__(self, brand):
        self.brand = brand

class Car(Vehicle):
    def __init__(self, brand, model):
        super().__init__(brand) # calls Vehicle's __init__
        self.model = model

c = Car("Toyota", "Corolla")
print(f"{c.brand} {c.model}") # Toyota Corolla
```

## 9.6 Dunder (Magic) Methods

### CODE

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __str__(self):
        # controls what print() shows
        return f"Point({self.x}, {self.y})"

    def __add__(self, other):
        # enables the + operator on Points
        return Point(self.x + other.x, self.y + other.y)

p1, p2 = Point(1, 2), Point(3, 4)
print(p1 + p2) # Point(4, 6)
```



## Practice Exercises — Object-Oriented Programming

1. **EASY** Create a `Rectangle` class with `width` and `height` attributes and methods `area()` and `perimeter()`.
2. **MEDIUM** Create a base class `Shape` with a method `area()` that raises `NotImplementedError`, then create `Circle` and `Square` subclasses that override it properly.
3. **MEDIUM** Create a `Student` class with a private attribute `__grades` (a list), plus methods `add_grade(g)` and `get_average()` that computes the average safely (handle the empty-list case).
4. **HARD** Build a small class hierarchy: `Employee` (base) → `Manager` and `Developer` (subclasses). Each should override a `calculate_bonus()` method differently, and you should create a list of mixed employees and print each one's bonus using polymorphism.

# 10

## **NumPy Fundamentals**

Fast numerical computing with arrays, broadcasting, and vectorized operations.

# 10. NumPy Fundamentals

## 10.1 What is NumPy?

NumPy (Numerical Python) is the foundational library for numerical computing in Python. Its core object is the `ndarray` — a fast, memory-efficient, multi-dimensional array that supports vectorized operations (no explicit loops needed).

### CODE — setup

```
import numpy as np
```

## 10.2 Creating Arrays

### CODE

```
a = np.array([1, 2, 3, 4])
b = np.array([[1, 2], [3, 4]])    # 2D array (matrix)

zeros = np.zeros((2, 3))         # 2x3 array of zeros
ones = np.ones((3, 3))          # 3x3 array of ones
identity = np.eye(3)             # 3x3 identity matrix
seq = np.arange(0, 10, 2)        # [0, 2, 4, 6, 8]
lin = np.linspace(0, 1, 5)       # 5 evenly spaced values 0 to 1

print(a.shape, a.dtype, a.ndim)  # (4,) int64 1
print(b.shape)                   # (2, 2)
```

## 10.3 Indexing and Slicing

### CODE

```
arr = np.array([10, 20, 30, 40, 50])
print(arr[1])          # 20
print(arr[1:4])         # [20 30 40]
print(arr[arr > 25])    # [30 40 50] — boolean ("fancy") indexing

matrix = np.array([[1, 2, 3], [4, 5, 6]])
print(matrix[0, 2])     # 3    (row 0, col 2)
print(matrix[:, 1])     # [2 5] — entire column 1
```

## 10.4 Vectorized Arithmetic (No Loops!)

### CODE

```
x = np.array([1, 2, 3])
y = np.array([10, 20, 30])

print(x + y)           # [11 22 33]
print(x * y)           # [10 40 90]
print(y / x)           # [10. 10. 10.]
print(x ** 2)          # [1 4 9]
print(np.sqrt(y))      # [3.162 4.472 5.477]
```

### WHY NUMPY IS FAST

NumPy operations run as compiled C code under the hood and operate on whole arrays at once ("vectorization"), avoiding slow Python-level `for` loops. This makes NumPy 10–100x faster than equivalent pure-Python loops for numerical work.

## 10.5 Broadcasting

Broadcasting lets NumPy perform operations on arrays of different (but compatible) shapes without explicitly copying data.

### CODE

```
matrix = np.array([[1, 2, 3], [4, 5, 6]])
print(matrix + 10)          # adds 10 to every element
print(matrix * np.array([1, 0, -1])) # multiplies each row by [1, 0, -1]
```

### OUTPUT

```
[[11 12 13]
 [14 15 16]]
[[ 1  0 -3]
 [ 4  0 -6]]
```

## 10.6 Aggregate Functions

### CODE

```
data = np.array([[4, 9, 2], [7, 1, 5]])

print(data.sum())          # 28 — total of all elements
print(data.sum(axis=0))    # [11 10  7] — sum down each column
print(data.sum(axis=1))    # [15 13] — sum across each row
print(data.mean())         # 4.666...
print(data.max(), data.min())
print(data.std())          # standard deviation
```

## 10.7 Reshaping Arrays

### CODE

```
arr = np.arange(12)          # [0 1 2 ... 11]
reshaped = arr.reshape(3, 4) # 3 rows, 4 columns
print(reshaped)
print(reshaped.flatten())    # back to 1D
print(reshaped.T)           # transpose (swap rows/cols)
```

### COMMON PITFALL

`.reshape()` requires the total number of elements to stay the same. `np.arange(10).reshape(3, 4)` raises a `ValueError` because  $10 \neq 3 \times 4$ .

### Practice Exercises — NumPy Fundamentals

- EASY** Create a 1D NumPy array of the numbers 1 to 20, then extract only the even numbers using boolean indexing.
- EASY** Create a 4x4 matrix of random integers between 1 and 100 (use `np.random.randint`) and compute its row sums, column sums, and overall mean.
- MEDIUM** Given two arrays representing students' marks in two subjects, compute the average mark per student (without using a Python `for` loop).
- HARD** Simulate rolling two dice 10,000 times using `np.random.randint(1, 7, size=10000)` for each die, compute their sum array, and calculate the percentage of rolls where the sum equals 7.

# 11

## Pandas Fundamentals

Series, DataFrames, filtering, grouping, and real-world data wrangling.

# 11. Pandas Fundamentals

## 11.1 What is Pandas?

Pandas is built on top of NumPy and provides two core data structures — the `Series` (1D labeled array) and the `DataFrame` (2D labeled table, like a spreadsheet) — designed for practical data analysis.

### CODE — setup

```
import pandas as pd
```

## 11.2 Series

### CODE

```
s = pd.Series([10, 20, 30, 40], index=["a", "b", "c", "d"])
print(s)
print(s["b"])      # 20 — label-based access
```

### OUTPUT

```
a    10
b    20
c    30
d    40
dtype: int64
20
```

## 11.3 Creating a DataFrame

### CODE

```
data = {
    "Name": ["Asha", "Ravi", "Meera", "John"],
    "Age": [23, 35, 29, 41],
    "City": ["Mumbai", "Delhi", "Pune", "Chennai"],
    "Salary": [55000, 62000, 58000, 71000]
}
df = pd.DataFrame(data)
print(df)
```

### OUTPUT

|   | Name  | Age | City    | Salary |
|---|-------|-----|---------|--------|
| 0 | Asha  | 23  | Mumbai  | 55000  |
| 1 | Ravi  | 35  | Delhi   | 62000  |
| 2 | Meera | 29  | Pune    | 58000  |
| 3 | John  | 41  | Chennai | 71000  |

## 11.4 Reading and Inspecting Data

### CODE

```
df = pd.read_csv("employees.csv")    # load from a CSV file

print(df.head())                    # first 5 rows
print(df.tail(3))                   # last 3 rows
print(df.info())                    # column types, non-null counts
print(df.describe())                 # statistical summary of numeric columns
print(df.shape)                     # (rows, columns)
print(df.columns)                   # column names
```

## 11.5 Selecting and Filtering Data

### CODE

```
print(df["Name"])           # select a single column (returns a Series)
print(df[["Name", "Age"]])  # select multiple columns
print(df.iloc[0])           # first row, by position
print(df.loc[0, "Name"])    # specific cell, by label

# Filtering rows with a boolean condition
high_earners = df[df["Salary"] > 58000]
print(high_earners)

# Combining conditions
result = df[(df["Age"] > 25) & (df["City"] == "Pune")]
print(result)
```

## 11.6 Adding, Modifying, and Dropping Columns

### CODE

```
df["Bonus"] = df["Salary"] * 0.1      # new column from existing one
df["Senior"] = df["Age"] > 30         # boolean column
df.drop("Bonus", axis=1, inplace=True) # remove a column
print(df)
```

## 11.7 Handling Missing Data

### CODE

```
print(df.isnull().sum())      # count missing values per column
df_clean = df.dropna()        # remove rows with any missing values
df_filled = df.fillna(0)       # replace missing values with 0
df["Age"] = df["Age"].fillna(df["Age"].mean()) # fill with column mean
```

## 11.8 Grouping and Aggregation

### CODE

```
city_avg = df.groupby("City")["Salary"].mean()
print(city_avg)

summary = df.groupby("City").agg(
    avg_salary=("Salary", "mean"),
    count=("Name", "count")
)
print(summary)
```

### OUTPUT

```
City
Chennai    71000.0
Delhi      62000.0
Mumbai     55000.0
Pune       58000.0
Name: Salary, dtype: float64
```

## 11.9 Sorting

### CODE

```
sorted_df = df.sort_values("Salary", ascending=False)
print(sorted_df)
```

## 11.10 Saving Data

### CODE

```
df.to_csv("output.csv", index=False)
df.to_excel("output.xlsx", index=False)
```

### TIP

Most Pandas operations return a **new** DataFrame by default rather than modifying in place. Use `inplace=True` only when you intentionally want to modify the original DataFrame directly.

### Practice Exercises — Pandas Fundamentals

1. **EASY** Create a DataFrame of 5 products with columns Name, Price, and Quantity. Add a new column Total = Price × Quantity.
2. **EASY** Load any CSV file and print its shape, column names, and the first 5 rows.
3. **MEDIUM** Given a DataFrame of student scores in 3 subjects, compute each student's average and add it as a new column. Then filter to show only students who scored above 75 on average.
4. **MEDIUM** Given a sales DataFrame with columns Region, Product, and Revenue, use `groupby` to find the total revenue per region, sorted from highest to lowest.
5. **HARD** Given a messy DataFrame with some missing values in the Age and Salary columns, write a cleaning pipeline that: fills missing Age with the median age, fills missing Salary with the mean salary per City group, and drops any remaining rows that still have missing values in other columns.